

Encuentro 10 — Listas y mutabilidad

Estructuras de datos y funciones · Unidad 2

1. Metadatos de bloque

Campo	Detalle
Encuentro	10 de 36
Unidad	2 — Estructuras de datos y funciones
Eje temático	2 — Estructuras de datos y funciones
Carácter/Objetivo	Conceptual
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Listas y mutabilidad
Requisitos previos	Variables y tipos (<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>), entrada por <code>input()</code> , conversión con <code>int()</code> / <code>float()</code> , estructura <code>if-else</code> , bucle <code>for</code> sobre rangos y cadenas, operadores <code>in</code> y <code>not in</code> . Todo visto en la Unidad 1.
Uso de celular	No permitido
Organización del trabajo	Individual con monitor, o en parejas si alcanzan las máquinas. Cada estudiante escribe su propio archivo <code>.py</code> . Rotación de monitor cada 20 minutos en parejas.

Reparto de tiempos

Momento	Tiempo
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

Al finalizar este encuentro vas a poder:

1. Crear una lista con literales y agregar elementos con `append` .
2. Acceder a elementos por índice y eliminar el último con `pop` .
3. Buscar la posición de un elemento con `index` y ordenar con `sort` .

4. Distinguir entre tipos mutables (lista) e inmutables (`str` , `tuple`).
5. Usar slicing `[i:j]` sobre listas y cadenas.

3. Teoría mínima (20 min)

Charla rápida: el pizarrón y la foto

Imaginá que tenés la lista de socios del club anotada en un pizarrón con tiza. Podés borrar un nombre, agregar otro al final, cambiar el orden. Eso es una **lista**: podés modificarla después de crearla. Ahora imaginá esa misma lista impresa en una foto: no podés tachar ni agregar nombres sin sacar otra foto. Eso es un **tipo inmutable** como el `str` o la `tuple`. Hoy manejamos el pizarrón.

Lo mínimo indispensable

Lista (`list`): secuencia mutable de elementos separados por coma entre corchetes.

```
member_names = ["Ana", "Luis", "Sofía"]
```

Métodos básicos (todas mutan la lista y devuelven `None`):

Operación	Código	Efecto
Agregar al final	<code>member_names.append("Pedro")</code>	Lista crece
Sacar el último	<code>popped = member_names.pop()</code>	Devuelve el elemento y lo borra
Buscar posición	<code>pos = member_names.index("Luis")</code>	Devuelve índice 0-based
Ordenar	<code>member_names.sort()</code>	Ordena ascendente, modifica la lista

Regla de oro: `append` , `pop` , `index` , `sort` no devuelven la lista. Esto:

```
# MAL - append devuelve None, la lista se pierde
member_names = member_names.append("Pedro")

# BIEN
member_names.append("Pedro")
```

Mutabilidad: una lista cambia “en el lugar”. Un `str` no:

```
name = "Ana"
# name[0] = "B" # TypeError: el str es inmutable
name = "Bet" + name[1:] # Hay que construir uno nuevo
```

Slicing `[i:j]`: obtiene un subsegmento desde `i` hasta `j-1`. Funciona en listas y strings. Índices fuera de rango no lanzan error, devuelven lo disponible.

```
items = [10, 20, 30, 40, 50]
sub = items[1:4] # [20, 30, 40]
first = items[:3] # [10, 20, 30]
last = items[-2:] # [40, 50]
```

4. Práctica guiada (35 min)

Armamos juntos un programa que administra la lista de socios del club de barrio.

Paso 1: crear el archivo `club_members.py` y escribir el programa completo.

```
# Lista inicial de socios
member_names = ["Ana García", "Luis Pérez", "Sofía Martínez", "Carlos López"]

print("=== Club de Barrio – Lista de socios ===")
print("Socios actuales:", member_names)

# --- Agregar un socio ---
print("\nAgregamos un nuevo socio...")
member_names.append("María Torres")
print("Después de append:", member_names)

# --- Buscar posición ---
print("\nBuscamos la posición de 'Luis Pérez'...")
position = member_names.index("Luis Pérez")
print(f"Luis Pérez está en la posición {position}")

# --- Sacar el último ---
print("\nEl secretario pidió borrar el último socio...")
removed = member_names.pop()
print(f"Sacamos a: {removed}")
print("Lista actual:", member_names)

# --- Ordenar alfabéticamente ---
print("\nOrdenamos la lista...")
member_names.sort()
print("Lista ordenada:", member_names)

# --- Slicing: primeros 2 socios ---
print("\nPrimeros 2 socios (slicing):", member_names[:2])

# --- Mostrar cantidad ---
print(f"\nTotal de socios: {len(member_names)}")
```

Paso 2: ejecutalo con `python club_members.py`.

Salida esperada:

```
=== Club de Barrio – Lista de socios ===
Socios actuales: ['Ana García', 'Luis Pérez', 'Sofía Martínez', 'Carlos López']

Agregamos un nuevo socio...
Después de append: ['Ana García', 'Luis Pérez', 'Sofía Martínez', 'Carlos López', 'María Torres']

Buscamos la posición de 'Luis Pérez'...
Luis Pérez está en la posición 1

El secretario pidió borrar el último socio...
Sacamos a: María Torres
Lista actual: ['Ana García', 'Luis Pérez', 'Sofía Martínez', 'Carlos López']

Ordenamos la lista...
Lista ordenada: ['Ana García', 'Carlos López', 'Luis Pérez', 'Sofía Martínez']

Primeros 2 socios (slicing): ['Ana García', 'Carlos López']

Total de socios: 4
```

Paso 3: probá mutabilidad. Agregá estas líneas al final y observá:

```
# Demostración de mutabilidad: La lista cambia "en el lugar"
original = member_names[:] # copia con slicing
member_names.append("Nuevo socio")
print(f"Lista original después de append: {original}") # no cambió
print(f"Lista actual: {member_names}")
```

5. Ejercicio independiente (25 min)

Consigna: Escribí un programa `member_ages.py` que trabaje con las edades de los socios del club.

1. Creá una lista llamada `ages` con las edades: 17, 15, 18, 16, 19.
2. Mostrá la lista completa.
3. Preguntá al usuario qué edad quiere agregar (usá `input()` + `int()` dentro de `try/except`).
4. Agregá la nueva edad con `append` y mostrá la lista actualizada.
5. Ordená la lista y mostrá el resultado.
6. Usá `pop()` para sacar la última edad y mostrá cuál se fue.
7. Mostrá la edad que está en la posición 2 (tercer elemento) usando índice.

Pista: recordá que `input()` devuelve `str`. Convertí con `int()` atrapando `ValueError`.

Solución: en el anexo docente.

6. Extensión y consolidación (20 min)

Si terminás el ejercicio base, probá estas variaciones:

1. **Edad máxima y mínima:** usá `max(ages)` y `min(ages)` para mostrar la edad más grande y la más chica.
2. **Slice de mayores:** mostrá las últimas 3 edades de la lista ordenada con slicing.
3. **Lista de nombres:** partiendo de `"Ana,Luis,Sofía,Carlos"` usá `split(",")` para generar una lista, agregale un nombre y mostrá el resultado.
4. **Mutabilidad vs string:** mostrá que no podés hacer `"Hola"[0] = "h"` y por qué.

7. Cierre (10 min)

Qué te llevás

- Las listas se crean con `[]`, se modifican con `append`, `pop`, `sort`.
- `append`, `pop`, `sort` **no devuelven la lista** — se usan solos en una línea.
- Los strings son inmutables: no podés cambiarles un carácter, tenés que construir uno nuevo.
- El slicing `[i:j]` funciona en listas y strings, y los índices fuera de rango no rompen.

Lo que viene

Encuentro 11: Tuplas y conjuntos (set)

8. Errores comunes y trampas

Error	Causa	Fix
<code>lista = lista.append(x)</code>	<code>append</code> devuelve <code>None</code> y muta la lista; reasignar la pierde	<code>lista.append(x)</code> en su propia línea
<code>lista = lista.sort()</code>	<code>sort()</code> muta y devuelve <code>None</code> , igual que <code>append</code>	<code>lista.sort()</code> solo, sin asignación
<code>s[0] = "H"</code> sobre un <code>str</code>	El <code>str</code> es inmutable — <code>TypeError</code>	Construir nueva cadena con slicing: <code>"H" + s[1:]</code>
Poner índice fuera de rango al acceder	Ejemplo: <code>items[10]</code> en lista de 5 elementos — <code>IndexError</code>	Verificar con <code>len(items)</code> o usar slicing que tolera rangos fuera de límite
Olvidar que <code>input()</code> devuelve <code>str</code>	Sumar <code>input()</code> + número: concatena en vez de sumar	Convertir con <code>int(input(...))</code> dentro de <code>try/except ValueError</code>
Confundir posición con contenido	<code>items.index(30)</code> busca el valor 30, no la posición 30	<code>index</code> busca por valor; para acceder por posición se usa <code>items[pos]</code>

Encuentro 11 — Tuplas y conjuntos (set)

Estructuras de datos y funciones · Unidad 2

1. Metadatos de bloque

Campo	Detalle
Encuentro	11 de 36
Unidad	2 — Estructuras de datos y funciones
Eje temático	2 — Estructuras de datos y funciones
Carácter/Objetivo	Procedimental
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Tuplas y conjuntos (set)
Requisitos previos	Listas: creación, <code>append</code> , <code>pop</code> , <code>index</code> , <code>sort</code> , acceso por índice, mutabilidad, slicing. Todo del encuentro 10.
Uso de celular	No permitido
Organización del trabajo	Individual con monitor, o en parejas. Cada estudiante escribe su propio <code>.py</code> .

Reparto de tiempos

Momento	Tiempo
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

Al finalizar este encuentro vas a poder:

1. Crear tuplas con `()` y explicar por qué son inmutables.
2. Crear conjuntos con `{}` y `set()`, y entender que no tienen duplicados ni orden fijo.
3. Elegir entre `list`, `tuple` y `set` según el problema (mutabilidad, unicidad, orden).
4. Usar `add`, `remove`, `in` y `len` sobre conjuntos.
5. Convertir entre tipos de colección según la necesidad.

3. Teoría mínima (20 min)

Charla rápida: la bandeja de herramientas

Imaginá que el club tiene tres tipos de registros:

- **Lista** (pizarrón): los socios activos, se modifican todos los días (mutables, orden importa).
- **Tupla** (foto impresa): las categorías de socio fijas — «menor», «adulto», «vitalicio». No se modifican nunca.
- **Set** (canasta de pelotas): los números de credenciales que ya se entregaron. No nos importa el orden, solo no repetir.

Cada estructura es la mejor para un trabajo distinto.

Lo mínimo indispensable

Tupla (`tuple`): secuencia **inmutable** entre paréntesis. No se puede modificar después de creada.

```
# Las categorías de socio siempre son las mismas
categories = ("menor", "adulto", "vitalicio")
print(categories[0])      # "menor" - acceso por índice sí funciona
# categories[0] = "infantil" # TypeError: tuple no admite asignación
```

Conjunto (`set`): colección **sin duplicados ni orden fijo**, entre llaves.

```
# Números de credencial ya entregados - sin repetir
issued_ids = {101, 105, 103, 101} # el 101 repetido se ignora
print(issued_ids) # {101, 103, 105} - orden no garantizado

issued_ids.add(110)
issued_ids.remove(105)
print(105 in issued_ids) # False
print(len(issued_ids)) # 3
```

❑ **Crear set vacío:** no se usa `{}` vacío (eso es un diccionario). Usar `set()`.

Conversión entre colecciones:

```
names_list = ["Ana", "Luis", "Ana"] # Lista con duplicado
names_set = set(names_list)         # {"Ana", "Luis"} - sin duplicados
names_unique_list = list(names_set) # ["Ana", "Luis"]
```

Criterio de elección:

Necesito	Uso	Razón
Modificar, orden importa	<code>list</code>	Mutable, ordenada
No modificar nunca	<code>tuple</code>	Inmutable, segura
Sin duplicados, orden no importa	<code>set</code>	Unicidad automática

4. Práctica guiada (35 min)

Armamos un programa que trabaja con datos del club usando las tres estructuras.

Paso 1: crear `club_collections.py` y escribir:

```

# === Tuplas: categorías fijas ===
categories = ("menor", "adulto", "vitalicio")
print("Categorías de socio (tupla):", categories)
print("Primera categoría:", categories[0])

# Intentar modificar (esto va a fallar, lo vemos comentado)
# categories[1] = "senior" # TypeError

# === Lista: socios activos ===
active_members = ["Ana García", "Luis Pérez", "Sofía Martínez"]
print("\nSocios activos (lista):", active_members)
active_members.append("Carlos López")
print("Después de agregar:", active_members)

# === Set: números de credencial ya emitidos ===
issued_ids = {101, 105, 103, 107}
print("\nCredenciales emitidas (set):", issued_ids)

# Agregar nuevas credenciales
issued_ids.add(110)
issued_ids.add(103) # duplicado, se ignora
print("Después de agregar 110 y 103 (duplicado):", issued_ids)

# Verificar si una credencial ya fue emitida
check_id = 103
if check_id in issued_ids:
    print(f"La credencial {check_id} ya fue emitida.")
else:
    print(f"La credencial {check_id} está disponible.")

# Eliminar una credencial
issued_ids.remove(107)
print("Después de eliminar 107:", issued_ids)

# === Conversión: lista de nombres sin duplicados ===
names_with_dupes = ["Ana", "Luis", "Ana", "Sofía", "Luis"]
unique_names = list(set(names_with_dupes))
print("\nNombres originales (con duplicados):", names_with_dupes)
print("Nombres únicos (set → list):", unique_names)

# === Mostrar cantidad de cada colección ===
print(f"\nCantidad de categorías: {len(categories)}")
print(f"Cantidad de socios activos: {len(active_members)}")
print(f"Cantidad de credenciales emitidas: {len(issued_ids)}")

```

Paso 2: ejecutá con `python club_collections.py`.

Salida esperada:

```

Categorías de socio (tupla): ('menor', 'adulto', 'vitalicio')
Primera categoría: menor

Socios activos (lista): ['Ana García', 'Luis Pérez', 'Sofía Martínez']
Después de agregar: ['Ana García', 'Luis Pérez', 'Sofía Martínez', 'Carlos López']

Credenciales emitidas (set): {101, 103, 105, 107}
Después de agregar 110 y 103 (duplicado): {101, 103, 105, 107, 110}
La credencial 103 ya fue emitida.
Después de eliminar 107: {101, 103, 105, 110}

Nombres originales (con duplicados): ['Ana', 'Luis', 'Ana', 'Sofía', 'Luis']
Nombres únicos (set → list): ['Ana', 'Luis', 'Sofía']

Cantidad de categorías: 3
Cantidad de socios activos: 4
Cantidad de credenciales emitidas: 4

```

5. Ejercicio independiente (25 min)

Consigna: Escribí un programa `club_registry.py` que:

1. Cree una tupla `months` con los nombres de los meses del año (no se modifica nunca).
2. Cree una lista `member_names` con los nombres: "Ana", "Luis", "Sofía", "Carlos", "Ana".
3. Convierta la lista a un set (`unique_names`) para eliminar el duplicado.
4. Agregue un nombre nuevo al set (pedilo por `input()`).
5. Muestre cuántos nombres únicos hay.
6. Pregunte un nombre y diga si está en el set de nombres únicos.

Pista: para que el set funcione bien, después de `set(lista)` usá `add()` para agregar y `in` para preguntar.

6. Extensión y consolidación (20 min)

1. **Unión de sets:** creá dos sets `set_a = {1, 2, 3}` y `set_b = {3, 4, 5}` . Mostrá la unión (`set_a | set_b`) y la intersección (`set_a & set_b`).
2. **Tupla como clave segura:** mostrá que una tupla puede usarse como elemento de un set (un `list` no puede).
3. **Eliminación segura en set:** usá `discard()` en vez de `remove()` para eliminar un elemento que quizá no existe (probá con uno existente y uno inexistente).

7. Cierre (10 min)

Qué te llevás

- **Tupla:** inmutable, para datos que no cambian. Acceso por índice igual que lista.
- **Set:** sin duplicados, sin orden fijo. Se crea con `{}` o `set()` . Set vacío solo con `set()` .
- Para decidir: ¿lo modifico? → `list` . ¿No cambia nunca? → `tuple` . ¿Solo quiero saber si está o no está? → `set` .
- `len()` funciona en las tres; `in` también.

Lo que viene

Encuentro 12: Diccionarios: clave y valor

8. Errores comunes y trampas

Error	Causa	Fix
<code>t[0] = x</code> sobre una tupla	La tupla es inmutable: <code>TypeError</code>	Usar lista si necesitás modificar, o crear una tupla nueva
Crear set vacío con <code>{}</code>	<code>{}</code> vacío es un diccionario, no un set	Usar <code>my_set = set()</code> para set vacío
Pensar que un set tiene orden	<code>set</code> no garantiza orden; en cada ejecución puede variar	Pasar a <code>list()</code> si necesitás orden, o usar <code>sorted()</code>
Usar <code>remove()</code> en un set con elemento inexistente	<code>KeyError</code>	Usar <code>discard()</code> que no lanza error si no existe, o verificar con <code>in</code> antes
Confundir <code>(1)</code> con una tupla	<code>(1)</code> es un <code>int</code> , no una tupla de un elemento	La tupla de un elemento necesita coma: <code>(1,)</code>
Meter una lista dentro de un set	Las listas no son "hashables": <code>TypeError</code>	Usar tupla en vez de lista dentro del set

Encuentro 12 — Diccionarios: clave y valor

Estructuras de datos y funciones · Unidad 2

1. Metadatos de bloque

Campo	Detalle
Encuentro	12 de 36
Unidad	2 — Estructuras de datos y funciones
Eje temático	2 — Estructuras de datos y funciones
Carácter/Objetivo	Procedimental
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Diccionarios: clave y valor
Requisitos previos	Listas, tuplas y conjuntos (set): creación, métodos, criterio de elección. Todo de los encuentros 10 y 11.
Uso de celular	No permitido
Organización del trabajo	Individual con monitor, o en parejas. Cada estudiante escribe su propio <code>.py</code> .

Reparto de tiempos

Momento	Tiempo
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

Al finalizar este encuentro vas a poder:

1. Crear un diccionario con pares clave-valor entre llaves.
2. Acceder a un valor por su clave con `[]` y `.get()`.
3. Agregar y modificar entradas, y eliminar con `pop` o `del`.
4. Recorrer un diccionario con `keys()`, `values()` e `items()`.
5. Prevenir `KeyError` usando `in` y `.get()`.

3. Teoría mínima (20 min)

Charla rápida: la ficha del socio

En el club de barrio, cada socio tiene una ficha con campos: nombre, edad, cuota al día, categoría. En Python, la ficha es un **diccionario**: una colección donde cada **clave** (nombre del campo) apunta a un **valor** (el dato). No importa el orden: busco por la clave y obtengo el valor al instante.

```
member = {
    "name": "Ana García",
    "age": 17,
    "fee_paid": True,
    "category": "menor"
}
```

Las listas funcionan con índices numéricos; los diccionarios con claves con nombre. Si la lista es una fila de casilleros numerados, el diccionario es un archivero con etiquetas.

Lo mínimo indispensable

Crear y acceder:

```
member = {"name": "Ana", "age": 17}
print(member["name"])    # Ana
print(member.get("age"))  # 17
print(member.get("email", "-")) # "-" - valor por defecto si no existe
```

❑ `member["clave_inexistente"]` lanza `KeyError`. Siempre verificar con `in` o usar `.get()`.

Agregar, modificar y eliminar:

```
member["phone"] = "11-5555-1234" # agregar nueva clave
member["age"] = 18                # modificar existente
del member["phone"]              # eliminar una clave
popped = member.pop("age")       # eliminar y devolver el valor
```

Recorridos:

```
# keys, values, items devuelven vistas (como sets)
for key in member.keys():
    print(key)

for value in member.values():
    print(value)

for key, value in member.items():
    print(f"{key}: {value}")
```

Diccionario vacío:

```
members = {} # diccionario vacío - esto SÍ es un dict
```

4. Práctica guiada (35 min)

Armamos un programa que administra los socios del club usando diccionarios.

Paso 1: crear `club_dict.py` y escribir:

```
# === Diccionario de un socio ===
member = {
    "name": "Ana García",
    "age": 17,
    "fee_paid": True,
    "category": "menor"
}

print("=== Ficha del socio ===")
# Acceso directo (sabemos que existe)
print("Nombre:", member["name"])
print("Edad:", member["age"])

# Acceso seguro con .get()
status = member.get("fee_paid", "No especificado")
print("Cuota al día:", status)

# === Agregar nueva clave ===
member["phone"] = "11-5555-1234"
print("\nDespués de agregar teléfono:")
print("Teléfono:", member["phone"])

# === Modificar un valor ===
member["age"] = 18
print("Edad actualizada:", member["age"])

# === Recorrer con items() ===
print("\n=== Todos los datos del socio ===")
for key, value in member.items():
    print(f"  {key}: {value}")

# === Prevenir KeyError ===
print("\n=== Búsqueda segura ===")
if "email" in member:
    print("Email:", member["email"])
else:
    print("El socio no tiene email registrado.")

# === Eliminar con pop ===
removed = member.pop("phone", None)
if removed:
    print(f"Teléfono eliminado: {removed}")

# === Mostrar claves y valores por separado ===
print("\nClaves del diccionario:", list(member.keys()))
print("Valores:", list(member.values()))
```

Paso 2: ejecutá con `python club_dict.py`.

Salida esperada:

```
=== Ficha del socio ===
Nombre: Ana García
Edad: 17
Cuota al día: True

Después de agregar teléfono:
Teléfono: 11-5555-1234
Edad actualizada: 18

=== Todos los datos del socio ===
name: Ana García
age: 18
fee_paid: True
category: menor
phone: 11-5555-1234

=== Búsqueda segura ===
El socio no tiene email registrado.

Claves del diccionario: ['name', 'age', 'fee_paid', 'category', 'phone']
Valores: ['Ana García', 18, True, 'menor', '11-5555-1234']
```

Paso 3: modificar un valor existente. Cambiá la categoría:

```
member["category"] = "adulto"
print("Nueva categoría:", member["category"])
```

5. Ejercicio independiente (25 min)

Consigna: Escribí un programa `members_directory.py` que administre una lista de socios.

1. Creá un diccionario vacío `members` (la clave será el nombre, el valor será la edad).
2. Agregá tres socios: Ana (17), Luis (15), Sofía (18) — asignando directamente con `members["Ana"] = 17`.
3. Mostrá todos los socios recorriendo con `.items()`.
4. Preguntá un nombre por `input()`. Si existe en el diccionario, mostrá su edad. Si no, decí que no está registrado.
5. Preguntá un segundo nombre y eliminálo del diccionario con `pop()` si existe, o mostrá un mensaje si no.

Pista: usá `in` para verificar si un nombre existe antes de acceder con `[]`. `.pop(clave, None)` también es seguro.

6. Extensión y consolidación (20 min)

1. **Lista de diccionarios:** creá una lista donde cada socio es un diccionario completo (nombre, edad, categoría, cuota). Recorrela y mostrá cada ficha.
2. **Actualizar valores:** pedí un nombre por `input()` y, si existe, modificá su edad.
3. **Contar categorías:** usando un diccionario adicional, contá cuántos socios hay en cada categoría.

7. Cierre (10 min)

Qué te llevás

- Los diccionarios guardan pares **clave** → **valor**. La clave es como el nombre del campo.
- Accedé con `dict["clave"]` si sabés que existe; usá `.get("clave", default)` si no estás seguro.
- `keys()`, `values()` e `items()` permiten recorrer el diccionario.
- Para agregar o modificar: `dict["clave"] = valor`. Para eliminar: `pop(clave)` o `del`.
- **Nunca** supongas que una clave existe: usá `in` o `.get()`.

Encuentro 13: Funciones y ámbito local

8. Errores comunes y trampas

Error	Causa	Fix
<code>member["email"]</code> sin verificar	La clave no existe → <code>KeyError</code>	Usar <code>if "email" in member:</code> o <code>member.get("email", "no tiene")</code>
Pensar que las claves tienen orden fijo	Dict preserva orden de inserción desde Python 3.7, pero no es el concepto central	Si necesitas orden alfabético, usá <code>sorted(member.keys())</code>
Confundir <code>{}</code> vacío con set	<code>{}</code> es diccionario, no set	Set vacío es <code>set()</code> ; <code>{}</code> es <code>dict</code> vacío
Usar lista como clave	Las claves deben ser inmutables (<code>str</code> , <code>int</code> , <code>tuple</code>)	Usar <code>str</code> o <code>int</code> como clave
Olvidar que <code>keys()</code> devuelve una vista, no una lista	<code>members.keys()[0]</code> no funciona	Convertir con <code>list(members.keys())</code>
Usar <code>del member["clave"]</code> sin verificar	Si la clave no existe → <code>KeyError</code>	Usar <code>pop("clave", None)</code> que es segura

Encuentro 13 — Funciones y ámbito local

Estructuras de datos y funciones · Unidad 2

1. Metadatos de bloque

Campo	Detalle
Encuentro	13 de 36
Unidad	2 — Estructuras de datos y funciones
Eje temático	2 — Estructuras de datos y funciones
Carácter/Objetivo	Procedimental
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Funciones y ámbito local
Requisitos previos	Listas, tuplas, sets, diccionarios: creación, métodos, recorridos. Todo de los encuentros 10-12.
Uso de celular	No permitido
Organización del trabajo	Individual con monitor, o en parejas. Cada estudiante escribe su propio <code>.py</code> .

Reparto de tiempos

Momento	Tiempo
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

Al finalizar este encuentro vas a poder:

1. Definir una función con `def`, parámetros y `return`.
2. Llamar a una función y usar su valor de retorno.
3. Explicar que las variables dentro de una función son locales y no existen afuera.
4. Escribir un archivo organizado con funciones arriba y `if __name__ == "__main__":` abajo.
5. Descomponer un problema pequeño en funciones.

3. Teoría mínima (20 min)

Charla rápida: la receta del cocinero

Una función es como una receta. Le pasás ingredientes (parámetros), seguís los pasos, y obtenés un plato (return). Mientras cocinás, usás tus propios bowls (variables locales) que no existen fuera de la cocina. Si necesitás algo del resultado, el cocinero te lo pasa en el plato.

Hasta ahora escribimos todo el código corrido, como una receta sin separar en pasos reutilizables. Con `def` aprendemos a empaquetar pasos para usarlos muchas veces.

Lo mínimo indispensable

Estructura de una función:

```
def nombre_funcion(parametro1, parametro2):  
    """Cuerpo: hace algo y devuelve un resultado."""  
    resultado = parametro1 + parametro2  
    return resultado
```

Ámbito local: las variables creadas dentro de la función **no existen fuera**.

```
def calculate_double(value):  
    double = value * 2 # double es local  
    return double  
  
result = calculate_double(5)  
print(result)          # 10  
# print(double)        # NameError - double no existe acá
```

Organización del archivo (desde hoy y siempre):

```
# 1. Primero todas las funciones  
def read_int(message):  
    while True:  
        try:  
            return int(input(message))  
        except ValueError:  
            print("Eso no es un número entero; intentá de nuevo.")  
  
def show_menu():  
    print("1. Ver socios")  
    print("2. Agregar socio")  
    print("3. Salir")  
  
# 2. Al final, el bloque de ejecución principal  
if __name__ == "__main__":  
    # El programa arranca acá  
    show_menu()  
    option = read_int("Opción: ")
```

¿Por qué `if __name__ == "__main__"`? Porque así el archivo solo se ejecuta cuando lo corremos directamente. Si lo importáramos desde otro archivo (en el futuro), las funciones estarían disponibles pero el bloque principal no se ejecutaría automáticamente.

Sin `return` la función devuelve `None` :

```
def greet(name):  
    print(f"Hola {name}")    # no hay return  
  
result = greet("Ana")  
print(result)    # None
```

Regla: si una función produce un valor, lo devuelve con `return`. Si solo hace algo (como imprimir), no necesita `return`, pero no esperes un valor útil de vuelta.

4. Práctica guiada (35 min)

Armamos un programa con funciones para el club.

Paso 1: crear `club_functions.py` y escribir:

```

# === FUNCIONES ===

def read_int(message):
    """Pide un número entero con validación (input devuelve str)."""
    while True:
        try:
            return int(input(message))
        except ValueError:
            print("Eso no es un número entero; intentá de nuevo.")

def add_member(members, name, age):
    """Agrega un socio al diccionario (clave=nombre, valor=edad)."""
    members[name] = age
    return members

def find_member(members, name):
    """Busca un socio por nombre. Devuelve la edad o None si no existe."""
    return members.get(name, None)

def show_all(members):
    """Muestra todos los socios con su edad."""
    if len(members) == 0:
        print("No hay socios registrados.")
        return

    print("=== Socios del club ===")
    for name, age in members.items():
        print(f"    {name}: {age} años")
    print(f"Total: {len(members)} socios")

# === BLOQUE PRINCIPAL ===
if __name__ == "__main__":
    club_members = {}

    add_member(club_members, "Ana García", 17)
    add_member(club_members, "Luis Pérez", 15)
    add_member(club_members, "Sofía Martínez", 18)

    show_all(club_members)

    print("\nAgregamos un nuevo socio...")
    add_member(club_members, "Carlos López", 16)
    show_all(club_members)

    print("\nBuscamos a 'Luis Pérez'...")
    age = find_member(club_members, "Luis Pérez")
    if age is not None:
        print(f"Luis Pérez tiene {age} años.")
    else:
        print("No encontrado.")

    print("\nBuscamos a 'María Torres'...")
    age = find_member(club_members, "María Torres")
    if age is not None:
        print(f"María Torres tiene {age} años.")
    else:
        print("No encontrado.")

```

Paso 2: ejecutá con `python club_functions.py`.

Salida esperada:

```

=== Socios del club ===
Ana García: 17 años
Luis Pérez: 15 años
Sofía Martínez: 18 años
Total: 3 socios

Agregamos un nuevo socio...
=== Socios del club ===
Ana García: 17 años
Luis Pérez: 15 años
Sofía Martínez: 18 años
Carlos López: 16 años
Total: 4 socios

Buscamos a 'Luis Pérez'...
Luis Pérez tiene 15 años.

Buscamos a 'María Torres'...
No encontrado.

```

Paso 3: comentá el `return members` dentro de `add_member`, volvé a ejecutar y observá que la función devuelve `None` — pero la lista se modificó igual porque `dict` es mutable. Comentá y preguntá: ¿es necesario ese `return`?

5. Ejercicio independiente (25 min)

Consigna: Escribí un programa `club_menu.py` con un menú de opciones usando funciones.

El programa debe tener estas funciones arriba del `if __name__ == "__main__":`:

1. `read_int(message)` — la función de validación de la práctica guiada.
2. `add_member(members, name, age)` — agrega un socio al diccionario.
3. `remove_member(members, name)` — elimina un socio del diccionario si existe. Devuelve `True` si lo eliminó, `False` si no.
4. `show_all(members)` — muestra la lista de socios.
5. `show_menu()` — imprime el menú.

En el bloque principal, el programa debe:

- Crear un diccionario vacío `members`.
- Mostrar el menú en un bucle: 1 = Ver todos, 2 = Agregar, 3 = Eliminar, 4 = Salir.
- Para agregar, pedir nombre y edad con `read_int`.
- Para eliminar, pedir nombre y llamar a `remove_member`.
- Salir cuando el usuario elija 4.

Pista: el bucle principal tiene que llamar a `show_menu()` y después leer la opción. Usá `while True` y `break` cuando la opción sea 4.

6. Extensión y consolidación (20 min)

1. **Función con tupla como retorno:** escribí una función `get_stats(ages)` que reciba una lista de edades y devuelva una tupla con `(min, max, promedio)`.
2. **Parámetro por defecto:** creá una función `create_member(name, age, category="menor")` que devuelva un diccionario de socio. Llamala sin pasar la categoría (usar el default) y con una categoría distinta.
3. **Docstring:** agregale un `"""docstring"""` a cada función explicando qué hace, qué parámetros recibe y qué devuelve.

7. Cierre (10 min)

Qué te llevás

- Las funciones se definen con `def nombre(parametros):` y se llaman con `nombre(args)`.
- Las variables dentro de la función son locales: no existen afuera.
- `return` devuelve un valor; sin `return` la función devuelve `None`.
- El archivo se organiza: **funciones primero**, `if __name__ == "__main__":` **al final**.
- Las colecciones mutables (listas, dicts) se modifican dentro de la función incluso sin `return`.

Lo que viene

Encuentro 14: Cierre U2: repaso y TP

8. Errores comunes y trampas

Error	Causa	Fix
Llamar a una función sin paréntesis	<code>result = show_all</code> asigna la función, no la ejecuta	Siempre <code>show_all()</code> con paréntesis para invocar
Usar variable local fuera de la función	<code>NameError</code> porque no existe en el ámbito global	Devolver el valor con <code>return</code> y capturarlo al llamar
Modificar un parámetro de tipo inmutable (int, str, tuple) y esperar cambio afuera	Los tipos inmutables no se modifican “en el lugar”; la función recibe una copia	Devolver el nuevo valor con <code>return</code> y reasignarlo
Olvidar <code>return</code> y usar el resultado igual	La función devuelve <code>None</code> → <code>result = greet("Ana"); print(result + 1)</code> falla	Agregar <code>return valor</code> en la función
Código suelto sin <code>if __name__ == "__main__":</code>	El archivo funciona igual por ahora, pero es mala práctica que se vuelve problema al importar	Mover la ejecución al bloque principal
Definir función después de usarla	Python lee el archivo de arriba a abajo — <code>NameError</code> si la función no está definida aún	Las funciones siempre arriba, la ejecución al final

Encuentro 14 — Cierre U2: repaso y TP

Estructuras de datos y funciones · Encuentro de cierre de unidad

1. Metadatos de bloque

Campo	Detalle
Encuentro	14 de 36
Unidad	2 — Estructuras de datos y funciones
Eje temático	2 — Estructuras de datos y funciones
Carácter/Objetivo	Actitudinal
Estructura	cierre
Duración teórica	120 minutos (2 horas reloj)
TP obligatorio	TP-U2: programa con colecciones y funciones
Concepto nuevo	Cierre U2: repaso y TP
Requisitos previos	Listas, tuplas, sets, diccionarios, funciones, ámbito local, <code>if __name__ == "__main__":</code> . Todo de los encuentros 10-13.
Uso de celular	No permitido
Organización del trabajo	Individual o en parejas (misma modalidad que el curso). Cada estudiante produce su propio archivo en su carpeta <code>tp-u2/</code> del repositorio grupal.

Reparto de tiempos

Momento	Tiempo
Apertura	10 min
Consolidación	40 min
Trabajo del TP	45 min
Ciclo de entrega	15 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

Al finalizar este encuentro vas a poder:

1. Sistematizar las cuatro colecciones de la Unidad 2 (list, tuple, set, dict) y saber cuándo usar cada una.
2. Descomponer un problema de dominio en funciones con `def`, parámetros y `return`.
3. Armar un programa completo con menú, colecciones y funciones.
4. Entregar el TP-U2 siguiendo la rutina de Git: carpeta nueva, commits y push.

3. Apertura (10 min)

Charla rápida: el club entero en un programa

En los últimos cuatro encuentros aprendiste cuatro herramientas para organizar datos:

- **Listas** — la guía de socios (se modifica, orden importa).
- **Tuplas** — las categorías fijas (no cambian nunca).
- **Sets** — las credenciales emitidas (sin duplicados).
- **Diccionarios** — la ficha de cada socio (clave → valor).

Y con funciones (`def`) aprendiste a empaquetar cada tarea en un bloque reutilizable.

Hoy juntamos todo en el **TP-U2**: un programa que administra los socios de un club de barrio, con menú, colecciones y funciones. Y lo entregás con carpeta nueva, commit y push, como ya sabés hacer desde el encuentro 8.

Mapa de la Unidad 2

Tipo	Colección	Métodos clave	Mutabilidad
Secuencia mutable	<code>list</code>	<code>append</code> , <code>pop</code> , <code>index</code> , <code>sort</code>	☐ Mutable
Secuencia inmutable	<code>tuple</code>	Acceso por índice	☐ Inmutable
Conjunto	<code>set</code>	<code>add</code> , <code>remove</code> , <code>discard</code> , <code>in</code>	☐ Mutable (elementos únicos)
Mapeo	<code>dict</code>	<code>keys</code> , <code>values</code> , <code>items</code> , <code>get</code> , <code>pop</code>	☐ Mutable

4. Consolidación (40 min)

Repaso exprés con ejercicios cortos

Escribí en un archivo `review_u2.py` y ejecutá cada fragmento. No copies todo de una vez: escribí, ejecutá, entendé.

1. Lista — filtrar mayores de 16:

```
ages = [17, 15, 18, 16, 19, 14]
old_enough = []
for age in ages:
    if age >= 16:
        old_enough.append(age)
print("Mayores o igual a 16:", old_enough)
```

2. Set — eliminar duplicados de una lista:

```
names = ["Ana", "Luis", "Ana", "Sofía", "Luis", "Carlos"]
unique = set(names)
print("Nombres únicos:", unique)
print("Cantidad de nombres distintos:", len(unique))
```

3. Diccionario — buscar y mostrar:

```

member = {"name": "Ana", "age": 17, "fee_paid": True}
search_key = input("Clave a buscar: ")

# Acceso seguro
value = member.get(search_key, "Clave no encontrada")
print(f"Valor de '{search_key}': {value}")

```

4. Tupla como estructura fija:

```

def get_coordinates():
    """Devuelve un par (x, y) como tupla."""
    return (10, 20)

point = get_coordinates()
print(f"Coordenada X: {point[0]}, Y: {point[1]}")

```

5. Función que usa un diccionario:

```

def count_by_category(members):
    """Recibe un dict nombre->categoría, devuelve dict conteo."""
    counts = {}
    for name, category in members.items():
        if category in counts:
            counts[category] += 1
        else:
            counts[category] = 1
    return counts

if __name__ == "__main__":
    data = {"Ana": "menor", "Luis": "menor", "Sofía": "adulto", "Carlos": "vitalicio"}
    result = count_by_category(data)
    print("Conteo por categoría:", result)

```

5. Trabajo del TP (45 min)

TP-U2: Administración del club de barrio

Dominio: club de barrio — socios, cuotas, categorías, credenciales.

Escribí un programa `main.py` dentro de una carpeta `tp-u2/` (creala si no existe) que tenga **funciones** arriba y **bloque principal** abajo, con un menú interactivo.

Requisitos funcionales:

- El programa arranca con algunos socios precargados (mínimo 3) en un diccionario donde cada clave es el nombre y cada valor es otro diccionario con edad, categoría y cuota al día (bool).
- Menú con estas opciones:
 - 1. Ver todos los socios:** muestra nombre, edad, categoría y estado de cuota.
 - 2. Agregar socio:** pide nombre, edad, categoría (menor/adulto/vitalicio) y si pagó (sí/no → bool).
 - 3. Buscar socio:** pide nombre, muestra todos sus datos o dice “no encontrado”.
 - 4. Dar de baja:** elimina un socio por nombre (con confirmación de si existía).
 - 5. Socios con cuota al día:** muestra solo los que pagaron.
 - 6. Salir.**
- Cada opción del menú debe estar implementada en una función separada.

- Validación de entrada numérica con `read_int` (como en el encuentro 13).
- El programa no usa archivos ni persistencia: los datos viven en memoria mientras se ejecuta.

Entrega: carpeta `tp-u2/` con el archivo `main.py`, commit con mensaje `tp-u2: programa con colecciones y funciones`, y push al repositorio grupal.

Pista: pensá la estructura de datos primero. Un diccionario de diccionarios te permite buscar por nombre sin recorrer:

```
club = {  
    "Ana García": {"age": 17, "category": "menor", "fee_paid": True},  
    "Luis Pérez": {"age": 15, "category": "menor", "fee_paid": False},  
    "Sofía Martínez": {"age": 18, "category": "adulto", "fee_paid": True}  
}
```

6. Ciclo de entrega (15 min)

Rutina de Git (recordatorio breve)

Ya la conocés del encuentro 8. Repaso rápido:

```
# 1. Asegurate de estar en la carpeta del repositorio  
cd /ruta/al/repo  
  
# 2. Creá la carpeta del TP (si no existe)  
mkdir tp-u2  
  
# 3. Copiá o creá tu main.py adentro  
  
# 4. Agregá todo y commit  
git add .  
git commit -m "tp-u2: programa con colecciones y funciones"  
  
# 5. Subí a GitHub  
git push
```

Antes del commit, verificá:

- ☐ El archivo compila con `python tp-u2/main.py`.
- ☐ El menú muestra las 6 opciones y responde a cada una.
- ☐ Probaste: agregar socio, buscar, dar de baja, filtrar por cuota al día.
- ☐ Probaste: poner texto donde va un número (el programa no debe romperse).
- ☐ Probaste: buscar un socio que no existe.

7. Cierre (10 min)

Qué te llevás de la Unidad 2

- Cuatro colecciones** para organizar datos: listas (mutables), tuplas (inmutables), sets (únicos), diccionarios (clave→valor).
- Funciones** para descomponer problemas: `def`, parámetros, `return`, ámbito local.
- Organización profesional:** funciones primero, `if __name__ == "__main__":` al final.
- Validación:** `try/except ValueError` para entrada numérica, `in` y `.get()` para diccionarios.
- Rutina Git:** carpeta por trabajo, commit por entrega, push a GitHub.

Lo que viene

Encuentro 15: Evaluación de la Unidad 2

8. Errores comunes y trampas

Error	Causa	Fix
Usar lista de diccionarios en vez de dict de dicts para búsqueda	Recorrer una lista para buscar un socio es más lento y verboso	Usar <code>dict</code> anidado: nombre como clave facilita búsqueda directa
No validar entrada numérica en edad	<code>input()</code> devuelve str; comparar con <code>int</code> no funciona o suma mal	Usar <code>read_int()</code> con <code>try/except ValueError</code>
Olvidar <code>break</code> en el menú	El bucle nunca termina aunque el usuario elija "Salir"	<code>while True + if opcion == 6: break</code>
Mezclar <code>{}</code> para set y dict	Creer que <code>{}</code> vacío es un set	<code>{}</code> es dict vacío; <code>set()</code> es set vacío
No manejar <code>KeyError</code> al buscar socio	Usar <code>club[nombre]</code> sin verificar <code>in</code>	Usar <code>.get(nombre)</code> o <code>if nombre in club:</code>
Asignar resultado de <code>append</code> o <code>sort</code>	<code>lista = lista.append(x)</code> → pierde la lista	Usar <code>lista.append(x)</code> en su propia línea

Evaluación de la Unidad 2 — Encuentro 15

Evaluación de la instancia «Evaluación de la Unidad 2 — Encuentro 15» · Curso: Programación en Python. Documento de **metadatos y acuerdos de la instancia**, en registro docente formal. El material del alumno es la versión `evaluacion-u2-version-a` y sus versiones equivalentes generadas; sus soluciones y criterios de corrección van en el anexo docente separado (`evaluacion-u2-version-a-anexo-docente.md`).

1. Identificación

Campo	Detalle
Instancia	Evaluación de la Unidad 2 — Encuentro 15
Unidad evaluada	2 — Estructuras de datos y funciones
Eje temático	2 — Estructuras de datos y funciones
Carácter/Objetivo	Evaluación práctica individual: verificar la capacidad de organizar datos con listas y diccionarios, y descomponer programas en funciones con parámetros y retorno.
Destinatarios	Todo el curso
Duración teórica	120 minutos (2 horas reloj)
Uso de celular	No permitido
Documentos de la instancia	<code>evaluacion-u2.md</code> · <code>evaluacion-u2-version-a.md</code> · <code>evaluacion-u2-version-a-anexo-docente.md</code> (más las versiones equivalentes B generadas desde la A)

2. Estructura del encuentro (120 min)

Momento	Tiempo	Qué ocurre
Apertura e instrucciones	10 min	El docente explica la dinámica: defensa del TP-U2 + prueba individual. Asigna versiones (A o B) por fila.
Defensa individual del TP (grupos)	30 min	Rotación de grupos: mientras un grupo rinde, el docente toma la defensa del TP a otro grupo.
Prueba práctica individual	70 min	Cada estudiante resuelve su ejercicio de forma individual.
Cierre y recogida	10 min	Entrega por Git (commit + push). El docente anuncia devolución para el encuentro siguiente.
Total	120 min	

3. Regla canónica de la instancia

La entrega del TP-U2 se verificó en el encuentro 14 (cierre de unidad). En este encuentro 15 se realiza la defensa individual del TP y la prueba práctica individual. La devolución de ambos se realiza al inicio del encuentro siguiente (encuentro 16). Si un estudiante no completa la prueba, entrega lo que tenga.

4. Defensa individual del TP (modalidad)

El docente se reúne con cada grupo por separado y verifica:

1. Repositorio en GitHub con carpeta `tp-u2/` y archivo `club.py` o similar.
2. Demo: el grupo ejecuta el programa y muestra las opciones del menú con colecciones y funciones.
3. Pregunta individual a cada integrante sobre una función específica (p. ej., «¿qué parámetros recibe esta función?», «¿qué devuelve?»).
4. Registro en planilla.

5. Alcance

Incluido: listas (`append`, `pop`, `index`, `sort`), tuplas, conjuntos (`set`), diccionarios (`keys`, `values`, `items`, `get`, `in`), funciones (`def`, parámetros, `return`), bloque `if __name__ == "__main__":`, menú en memoria, validación `try/except`.

Excluido: archivos, POO, lambda, comprehensions avanzadas.

Objetivo mínimo: escribir un programa que use una lista o diccionario para almacenar datos y funciones para descomponer la lógica, con menú interactivo y entrada validada.

6. Prueba práctica individual (versiones equivalentes)

Un único ejercicio en un archivo `.py`. Puntaje total: 100 puntos.

- **Parte 1 — Datos y funciones de base (30 pts):** diccionario/listas de ejemplo, funciones para listar y buscar.
- **Parte 2 — Funcionalidad principal (40 pts):** función que procese datos (registrar, modificar, eliminar) con validación.
- **Parte 3 — Reporte o resumen (20 pts):** función que muestre un resumen o estadística.
- **Parte 4 — Ítems conceptuales (10 pts).**

7. Criterios de calificación

Componente	Puntaje	Qué se observa
Estructura y convenciones	10 pts	Funciones arriba, <code>if __name__</code> , f-strings, snake_case, mensajes en español
Parte 1 — Datos y funciones base	30 pts	Colecciones bien definidas, funciones con parámetros y retorno
Parte 2 — Funcionalidad principal	40 pts	Validación, uso de colecciones, funciones bien diseñadas
Parte 3 — Reporte o resumen	10 pts	Salida clara, formato con f-strings
Ítems conceptuales	10 pts	Respuestas correctas

8. Condiciones de resolución de la prueba

Individual, con apuntes propios, sin celular, sin comunicación entre estudiantes. Convenciones del curso obligatorias. Entrega por Git en carpeta `evaluacion-u2/`.

9. Regla de equivalencia entre versiones

Misma estructura, mismos requisitos. Cambia el dominio (club de barrio ↔ taller de escuela) y sus tokens. Se corrigen con la misma tabla de puntaje.

10. Mecánica de asignación de versiones

Fila impar → versión A (club). Fila par → versión B (taller). Registro en planilla.

11. Devolución

Al inicio del encuentro 16. Estudiantes con menos de 60 pts recuperan al inicio del encuentro 17.

Evaluación de la Unidad 2 — Encuentro 15 — Versión A

Dominio de esta versión: **club de barrio** (gestión de socios y cuotas). Duración: 70 minutos. Puntaje total: 100 puntos.
Resolución individual, con computadora, sin celular. Las condiciones completas están en `evaluacion-u2.md`.

Antes de empezar

- Creá una carpeta `evaluacion-u2/` dentro de tu repositorio grupal.
- Dentro, creá un único archivo `club.py`.
- Seguí las convenciones del curso: identificadores en inglés snake_case, comentarios y mensajes en español, f-strings, `if __name__ == "__main__":`, sin persistencia.
- Toda entrada numérica debe validarse con `while True` + `try/except ValueError`.
- Al terminar: `git add .`, `git commit -m "evaluacion-u2: entrega"`, `git push`.

Objetivos de la prueba

1. Usar listas y diccionarios para modelar datos de un dominio.
2. Definir funciones con parámetros y valor de retorno.
3. Validar entrada del usuario con `try/except`.
4. Organizar el programa en un menú interactivo.

Material provisto

No hay esqueleto. El estudiante escribe todo desde cero.

Parte 1 — Datos y funciones de base (30 puntos)

Definí una lista `memberList` de socios del club. Cada socio es un diccionario con `memberName`, `memberAge` y `feePaid` (bool).

Ítem	Consigna	Pts
1a	Crear <code>memberList</code> con al menos 4 socios de ejemplo	10
1b	Función <code>list_members()</code> que muestre todos los socios con su estado de cuota	10
1c	Función <code>find_member(name)</code> que busque por nombre (insensible a mayúsculas) y devuelva el socio o <code>None</code>	10
Total		30

Parte 2 — Gestión de cuotas (40 puntos)

Ítem	Consigna	Pts
2a	Menú principal con opciones: 1-Listar, 2-Pagar cuota, 3-Agregar socio, 4-Mostrar estadísticas, 5-Salir	5
2b	Función <code>pay_fee(name)</code> : busca al socio por nombre, si existe y <code>feePaid</code> es <code>False</code> , lo cambia a <code>True</code> ; si ya pagó, avisa. Toda entrada validada.	15
2c	Función <code>add_member(name, age)</code> : agrega un nuevo socio con <code>feePaid = False</code> . Validar que no exista ya por nombre.	15
2d	Opción inválida → mensaje claro y vuelve al menú	5
Total		40

Parte 3 — Estadísticas (20 puntos)

Ítem	Consigna	Pts
3a	Función <code>show_stats()</code> : mostrar cantidad de socios totales, cuántos pagaron la <code>clubFee</code> y cuántos deben	10
3b	Función <code>active_members()</code> : mostrar solo los socios con cuota al día	10
Total		20

Parte 4 — Ítems conceptuales (10 puntos)

Al final del archivo, en comentarios:

Ítem	Pregunta	Pts
4a	¿Qué ventaja tiene usar un diccionario para cada socio en lugar de listas paralelas?	5
4b	¿Qué devuelve una función si no tiene <code>return</code> ?	5

Batería de verificación

Prueba	Pedido	Salida esperada
Menú	Programa arranca	Menú con 5 opciones
Opción 1	Listar	Muestra todos los socios con nombre, edad y estado de cuota
Opción 2	"Ana", paga	Cambia a pagado, muestra confirmación
Opción 2	"Ana" otra vez	"El socio ya tiene la cuota al día"
Opción 2	"Inexistente"	"Socio no encontrado."
Opción 3	"Nuevo", 16	Agrega, muestra confirmación
Opción 3	"Ana", 17	"Ya existe un socio con ese nombre."
Opción 4	Estadísticas	Muestra total, pagaron, deben
Opción 4	activos	Solo socios con cuota al día
Opción 5	Salir	"Saludos!"
Opción inválida	"9"	"Opción inválida."

Al terminar

Verificá que el archivo se ejecuta: `python club.py` . Luego commit y push.

Evaluación de la Unidad 2 — Encuentro 15 — Versión A

Dominio de esta versión: **taller de barrio** (gestión de herramientas y cuotas). Duración: 70 minutos. Puntaje total: 100 puntos. Resolución individual, con computadora, sin celular. Las condiciones completas están en `evaluacion-u2.md`.

Antes de empezar

- Creá una carpeta `evaluacion-u2/` dentro de tu repositorio grupal.
- Dentro, creá un único archivo `taller.py`.
- Seguí las convenciones del curso: identificadores en inglés snake_case, comentarios y mensajes en español, f-strings, `if __name__ == "__main__":`, sin persistencia.
- Toda entrada numérica debe validarse con `while True` + `try/except ValueError`.
- Al terminar: `git add .`, `git commit -m "evaluacion-u2: entrega"`, `git push`.

Objetivos de la prueba

1. Usar listas y diccionarios para modelar datos de un dominio.
2. Definir funciones con parámetros y valor de retorno.
3. Validar entrada del usuario con `try/except`.
4. Organizar el programa en un menú interactivo.

Material provisto

No hay esqueleto. El estudiante escribe todo desde cero.

Parte 1 — Datos y funciones de base (30 puntos)

Definí una lista `toolList` de herramientas del taller. Cada herramienta es un diccionario con `toolName`, `toolCondition` y `feePaid` (bool).

Ítem	Consigna	Pts
1a	Crear <code>toolList</code> con al menos 4 herramientas de ejemplo	10
1b	Función <code>list_members()</code> que muestre todos los herramientas con su estado de depósito	10
1c	Función <code>find_member(name)</code> que busque por nombre (insensible a mayúsculas) y devuelva el herramienta o <code>None</code>	10
Total		30

Parte 2 — Gestión de cuotas (40 puntos)

Ítem	Consigna	Pts
2a	Menú principal con opciones: 1-Listar, 2-Pagar depósito, 3-Agregar herramienta, 4-Mostrar estadísticas, 5-Salir	5
2b	Función <code>pay_fee(name)</code> : busca al herramienta por nombre, si existe y <code>feePaid</code> es <code>False</code> , lo cambia a <code>True</code> ; si ya pagó, avisa. Toda entrada validada.	15
2c	Función <code>add_member(name, age)</code> : agrega un nuevo herramienta con <code>feePaid = False</code> . Validar que no exista ya por nombre.	15
2d	Opción inválida → mensaje claro y vuelve al menú	5
Total		40

Parte 3 — Estadísticas (20 puntos)

Ítem	Consigna	Pts
3a	Función <code>show_stats()</code> : mostrar cantidad de herramientas totales, cuántos pagaron la <code>toolDeposit</code> y cuántos deben	10
3b	Función <code>active_members()</code> : mostrar solo los herramientas con depósito al día	10
Total		20

Parte 4 — Ítems conceptuales (10 puntos)

Al final del archivo, en comentarios:

Ítem	Pregunta	Pts
4a	¿Qué ventaja tiene usar un diccionario para cada herramienta en lugar de listas paralelas?	5
4b	¿Qué devuelve una función si no tiene <code>return</code> ?	5

Batería de verificación

Prueba	Turno	Salida esperada
Menú	Programa arranca	Menú con 5 opciones
Opción 1	Listar	Muestra todos los herramientas con nombre, edad y estado de depósito
Opción 2	"Ana", paga	Cambia a pagado, muestra confirmación
Opción 2	"Ana" otra vez	"El herramienta ya tiene la depósito al día"
Opción 2	"Inexistente"	"Herramienta no encontrado."
Opción 3	"Nuevo", 16	Agrega, muestra confirmación
Opción 3	"Ana", 17	"Ya existe un herramienta con ese nombre."
Opción 4	Estadísticas	Muestra total, pagaron, deben
Opción 4	activos	Solo herramientas con depósito al día
Opción 5	Salir	"Saludos!"
Opción inválida	"9"	"Opción inválida."

Al terminar

Verificá que el archivo se ejecuta: `python taller.py` . Luego commit y push.